



CentraleSupélec

SOFTWARE ENGINEERING PROJECT 2026

Supermarket Checkout System

Andrew Nohra & Saadeddine Dakdouki

Course: Object-Oriented Software Engineering

CentraleSupélec

June 2026

Contents

1	Introduction	2
2	Architecture & Design Decisions	3
2.1	Overall Architecture — The Facade	3
2.2	Authentication & Role Management — Single Responsibility Principle	3
2.3	Pricing & Discount Architecture — R5, R6	3
2.4	Delivery & Logistics — R10	3
3	Applied Design Patterns	4
3.1	Strategy Pattern — Customer Plans (R5b, R8b)	4
3.2	Observer Pattern — Inventory Alerts (R9)	5
4	Design Analysis	6
4.1	Advantages	6
4.2	Limitations & Future Work	6
5	UML Diagrams	7
5.1	Complete Class Diagram	7
5.2	Use Case Diagram	8
5.3	Sequence Diagrams	9
5.3.1	Checkout and Payment	9
5.3.2	Inventory Update and Low-Stock Alert	10
6	Configuration & Testing	11
6.1	Configuration File (<code>my_supermarket.ini</code>)	11
6.2	Test Scenarios	11
6.2.1	<code>testScenario1.txt</code>	11
6.3	JUnit Test Suite	11
7	Workload Split	12
8	Conclusion	12
A	Appendix A: Complete Class Diagram	13

Introduction

This project consists of designing and implementing a Java-based command-line supermarket checkout system that satisfies a complete set of functional requirements, ranging from basic checkout and bank payment (R1–R4) to advanced discount plans, home delivery, real-time inventory management, and dynamic logistics (R5–R10). The system supports user authentication, item registration, customer management, checkout sessions, bill computation, POS payment simulation, low-stock alerts, and fully automated test scenarios executed through a command-line interface.

The implementation was structured with a deliberate architectural philosophy centred on **high cohesion and low coupling**. The command-line interface layer is strictly separated from the business logic, and Gang of Four (GoF) design patterns were treated as foundational elements of the design rather than additions applied after the fact. This approach made it possible to satisfy the extensibility requirements (R5b, R6b) without modifying existing classes, and to add advanced features such as dynamic delivery pricing (R10) and Observer-based inventory alerts (R9) without restructuring the core architecture.

The implementation is divided into several packages in order to separate responsibilities: `model`, `system`, `cli`, `discount`, `payment`, `delivery`, and `inventory`.

The remainder of this report is organized as follows. Section 2 describes the key architectural decisions and how each one maps to the project requirements. Section 3 details the two design patterns applied, the Strategy and the Observer, and explains precisely which problem each one solves. Section 4 provides a critical analysis of the advantages and limitations of our solution. Section 5 presents the UML diagrams of the system. Section 6 documents the configuration file, test scenarios, and JUnit test suite. Section 7 presents the workload split between the two team members. Section 8 concludes the report with a reflection on design choices and potential extensions.

Architecture & Design Decisions

Overall Architecture — The Facade

The system is built around a strict two-layer separation. The `SupermarketCLI` class forms the interface layer: it reads commands from standard input, parses arguments using `Regex`, validates their count, converts strings to numbers where needed, and delegates execution to `SupermarketSystem`. The CLI contains no business logic whatsoever.

`SupermarketSystem` acts as a Facade and Controller, hiding the complexity of the domain models — `Cart`, `POS`, `Item`, `InventoryService` — from the CLI. It coordinates all major use cases: user authentication, item and customer registration, checkout creation, item scanning, bill computation, payment, delivery, inventory update, and revenue tracking. This design keeps the CLI simple and makes the business logic independently testable through JUnit without requiring any terminal simulation.

The full class structure is presented in the UML class diagram of Section 5.

Authentication & Role Management — Single Responsibility Principle

Instead of embedding authentication inside `SupermarketSystem`, user state and role verification were extracted into a dedicated `AuthManager` class. This enforces the Single Responsibility Principle: `SupermarketSystem` coordinates use cases, while `AuthManager` handles sessions and access control.

In practice, methods such as `requireManager()` and `requireCashier()` evaluate the `currentUser` and throw an `IllegalStateException` immediately if the role does not match, blocking unauthorized commands before any business logic runs. For example, a cashier attempting to call `addItem()` is rejected at the gate without reaching the catalogue. Three roles are supported — Manager, Cashier, and Customer — each mapped to a strict and non-overlapping set of CLI commands.

Pricing & Discount Architecture — R5, R6

The bill computation pipeline is a central design concern, as it must correctly stack two independent layers of discount without applying them out of order. The exact sequence implemented inside `SupermarketSystem.computeBill()` is as follows:

1. `cart.computeRawTotal()` — sums item prices before any reduction.
2. `cart.computeTotal()` — applies category-level discounts internally per item (R6), for example a 10% reduction on fresh vegetables or 5% on dairy.
3. `checkoutCustomer.getPlan().applyDiscount(...)` — applies the customer tier discount through the `DiscountPlan` Strategy interface (R5).
4. `plan.applyDeliveryDiscount(...)` — applies the plan-specific delivery reduction, if a delivery was requested (R8b).

This ordering ensures that category discounts are computed on raw item prices and that customer plan discounts are applied to the already-reduced subtotal, which is the correct commercial behaviour. The extensibility of both layers — new plans (R5b) and new categories (R6b) — is addressed through the Strategy pattern described in Section 3.

Delivery & Logistics — R10

Home delivery extends the base R7/R8 logic with dynamic time-slot management implemented through the `DeliveryService` and `DeliverySlot` classes. Each `DeliverySlot` represents a 2-hour delivery window. When a customer requests a slot, the system first calls `slot.hasCapacity()` and throws an `IllegalStateException` if the vehicle is already fully booked, preventing any overbooking.

Dynamic pricing is then applied based on the nature of the slot. If `slot.isPeak()` is true, a \$5.00 surcharge is added to the base delivery fee to reflect higher demand during peak hours. Conversely, if `slot.isEcoFriendly()` is true — meaning a delivery truck is already scheduled in the customer's vicinity — a \$3.00 discount is applied to incentivise sustainable slot choices. Finally, if the cart weight exceeds 50 kg, the system throws an `IllegalStateException` and refuses the delivery entirely, as required by R8.

Plan-based delivery discounts (R8b) are applied as a final step through the `DiscountPlan` Strategy: prime customers pay 50% of the computed delivery fee, and platinum customers receive free delivery.

Applied Design Patterns

In our implementation, we applied two Gang of Four design patterns as foundational architectural choices: the Strategy pattern for discount and pricing extensibility, and the Observer pattern for inventory notifications. The Facade role of `SupermarketSystem` is described in Section 2.

Strategy Pattern — Customer Plans (R5b, R8b)

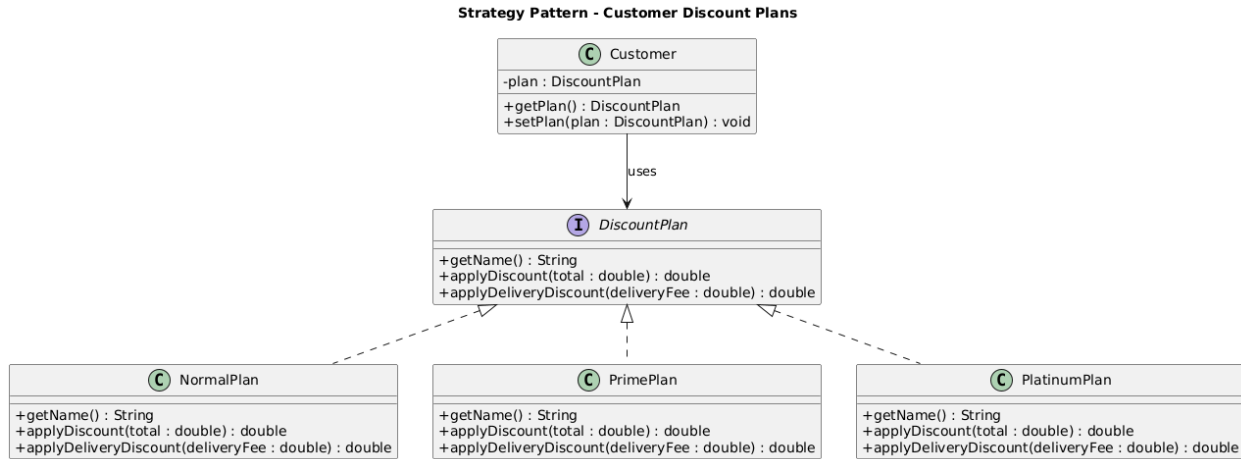


Figure 1: Strategy pattern applied to customer discount plans.

Problem. The system must support multiple customer discount plans with different pricing rules (R5), and the supermarket owner must be able to introduce new plans or modify existing ones without touching the checkout logic (R5b). Hard-coding `if/else` chains for each plan inside `computeBill()` would violate the Open/Closed Principle and make every future change a source of regression risk.

Structure. The `DiscountPlan` interface defines two methods: `applyDiscount` for item-level reductions and `applyDeliveryDiscount` for delivery fee reductions. Three concrete classes implement this interface: `NormalPlan`, `PrimePlan`, and `PlatinumPlan`.

Behaviour. `NormalPlan` returns amounts unchanged. `PrimePlan` checks if `(amount >= 50.0)` before returning `amount * 0.8`, applying the 20% discount only when the item total meets the threshold. `PlatinumPlan` unconditionally returns `amount * 0.7`, applying a 30% discount regardless of the total. The same interface governs delivery: a prime customer pays `deliveryFee * 0.5`, and a platinum customer pays nothing.

Extensibility. Adding a future “Gold Plan” requires only creating a new class implementing `DiscountPlan`. The `computeBill()` method, the `Cart`, and the POS remain completely unchanged. This directly satisfies R5b and R6b with no modification of existing code.

This pattern is also applied to category-level pricing policies (R6). Each `Category` holds a configurable discount percentage, making it straightforward to introduce new categories or adjust existing reductions through the `setCategoryDiscount` command.

Observer Pattern — Inventory Alerts (R9)

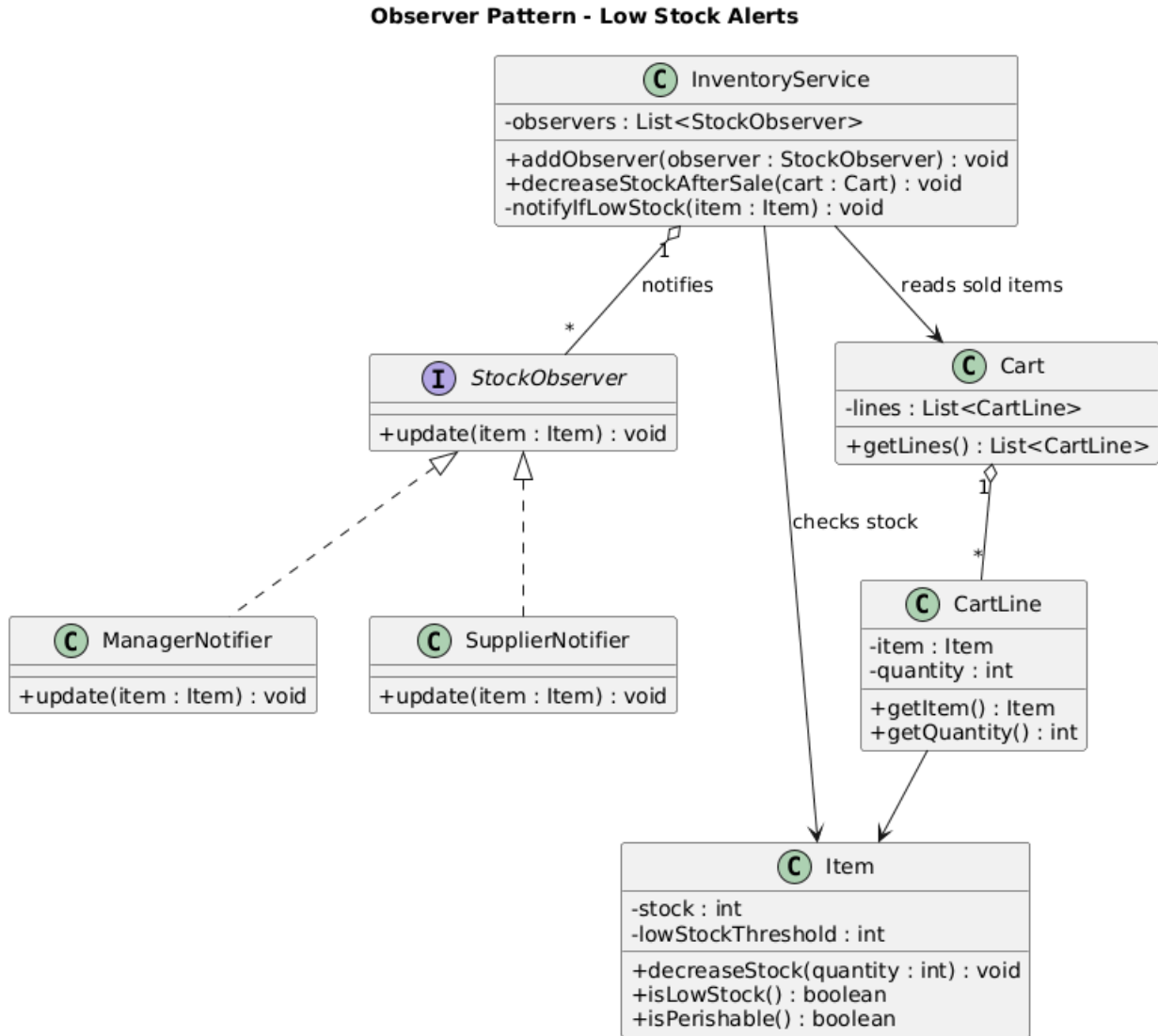


Figure 2: Observer pattern applied to low-stock inventory alerts.

Problem. After a successful sale, the system must notify a Manager or Supplier module whenever perishable items fall below a configurable threshold (R9). Implementing this notification inside the payment or inventory update code would tightly couple unrelated responsibilities, making the system harder to extend and test.

Structure. *InventoryService* is the Observable Subject. It maintains a list of registered observers implementing the *StockObserver* interface, which defines a single `update(Item item)` method. Two concrete observers are provided: *ManagerNotifier* and *SupplierNotifier*, each printing an alert with the item name and current stock level.

Trigger mechanism. Inside `decreaseStockAfterSale()`, the inventory service iterates over each *CartLine*, decrements the stock by the purchased quantity, and then evaluates two conditions. First, it checks whether the item is perishable — in our implementation, meat and dairy products are classified as perishable. Second, it checks whether the updated stock is at or below the item’s configured low-stock threshold. If both conditions hold, `notifyObservers(item)` is called, which invokes `update(item)` on every registered observer.

Critically, this entire flow is triggered only after a successful POS payment. A refused transaction never calls `decreaseStockAfterSale()`, so inventory and notifications remain unaffected by failed payments.

Extensibility. Adding an email notifier, an SMS alert, or a dashboard indicator requires only implementing *StockObserver* and registering the new observer with *InventoryService*. No existing class needs modification.

Design Analysis

Advantages

Testability. The strict separation between the CLI and `SupermarketSystem` means the entire business logic can be tested through JUnit without simulating terminal input. Each domain class is independently testable, which is why our JUnit suite covers all significant methods across all packages.

Open/Closed compliance. Both extensibility requirements — R5b (new discount plans) and R6b (new item categories) — are satisfied without modifying any existing class. New plans require a new `DiscountPlan` implementation; new categories are created on the fly by the `addItem` command. The Observer pattern offers the same guarantee for notification targets.

Clarity of responsibilities. Each package has a single, well-defined concern. The `cli` package handles input parsing. The `system` package coordinates use cases. The `discount`, `delivery`, `payment`, and `inventory` packages each own their domain. This makes the codebase navigable and reduces the risk of unintended side effects when modifying one area.

Reproducibility. The combination of `my-supermarket.ini` for default setup and `simulatePayment` for forced payment outcomes means every test scenario is fully deterministic and reproducible without any manual state manipulation.

Limitations & Future Work

In-memory storage. All data — users, items, inventory, revenue — is stored in `HashMap` structures that are lost when the application terminates. In a production environment, `AuthManager` and `InventoryService` would be connected to a persistent SQL database via JDBC, allowing state to survive across sessions and enabling multi-instance deployments.

Simplified geolocation. In the current implementation, the delivery distance is hardcoded as `distanceKm = 10.0` inside `SupermarketSystem.requestDelivery()`. The `address` string provided by the customer is stored but not processed. A production version would integrate a Maps API to dynamically compute the real distance between the supermarket and the delivery address.

No concurrency support. The system assumes a single active cashier session at any given time. Shared mutable state such as inventory counts and the revenue counter are not thread-safe. A real deployment with multiple concurrent checkout terminals would require transaction locking or an optimistic concurrency model.

UML Diagrams

Complete Class Diagram

Due to its size and the comprehensive nature of the system, the full UML class diagram is provided in **Appendix A** at the end of this document.

The complete UML class diagram gives a global view of the system architecture, illustrating how the main classes interact across the following packages:

- **cli**: Contains the command-line interface, whose sole responsibility is parsing input and delegating execution to **SupermarketSystem**.
- **system**: Contains **SupermarketSystem**, the central Facade and Controller of the application, along with the **AuthManager**.
- **model**: Contains the core business entities, including **User** (parent of **Manager**, **Cashier**, and **Customer**), **Item**, **Category**, **Cart**, and **CartLine**.
- **discount**: Contains the **DiscountPlan** Strategy hierarchy for customer tiers.
- **payment**: Models the POS terminal and the **TransactionAuthorizationSystem**.
- **delivery**: Handles dynamic logistics through **DeliveryService** and **DeliverySlot**.
- **inventory**: Manages the **InventoryService** and the **StockObserver** alert chain.

The diagram highlights three main structural points: the **DiscountPlan** Strategy hierarchy radiating from its interface, the **StockObserver** chain registered on the **InventoryService**, and the isolation of **AuthManager** as a standalone class separate from **SupermarketSystem**.

While Appendix A provides the granular details of every class and attribute, understanding the macro-level dependencies is equally important. Below is the high-level architectural overview demonstrating the relationships between the system's core packages:

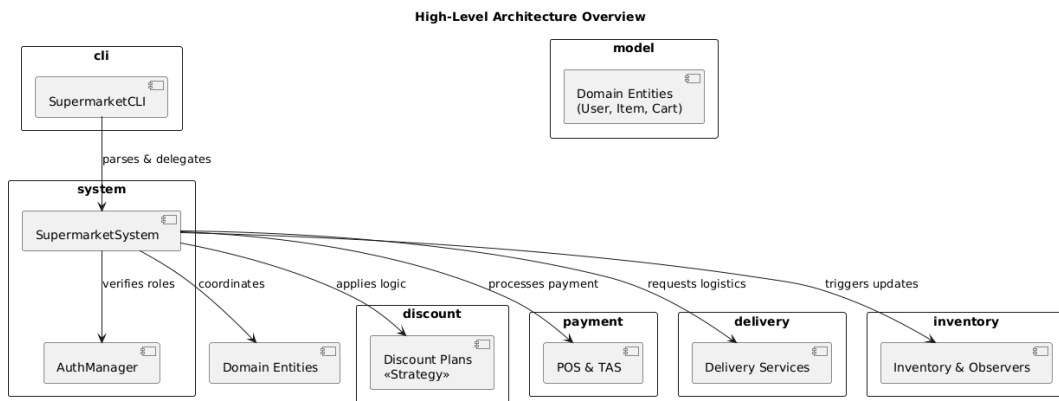


Figure 3: High-level package architecture of the supermarket checkout system.

Use Case Diagram

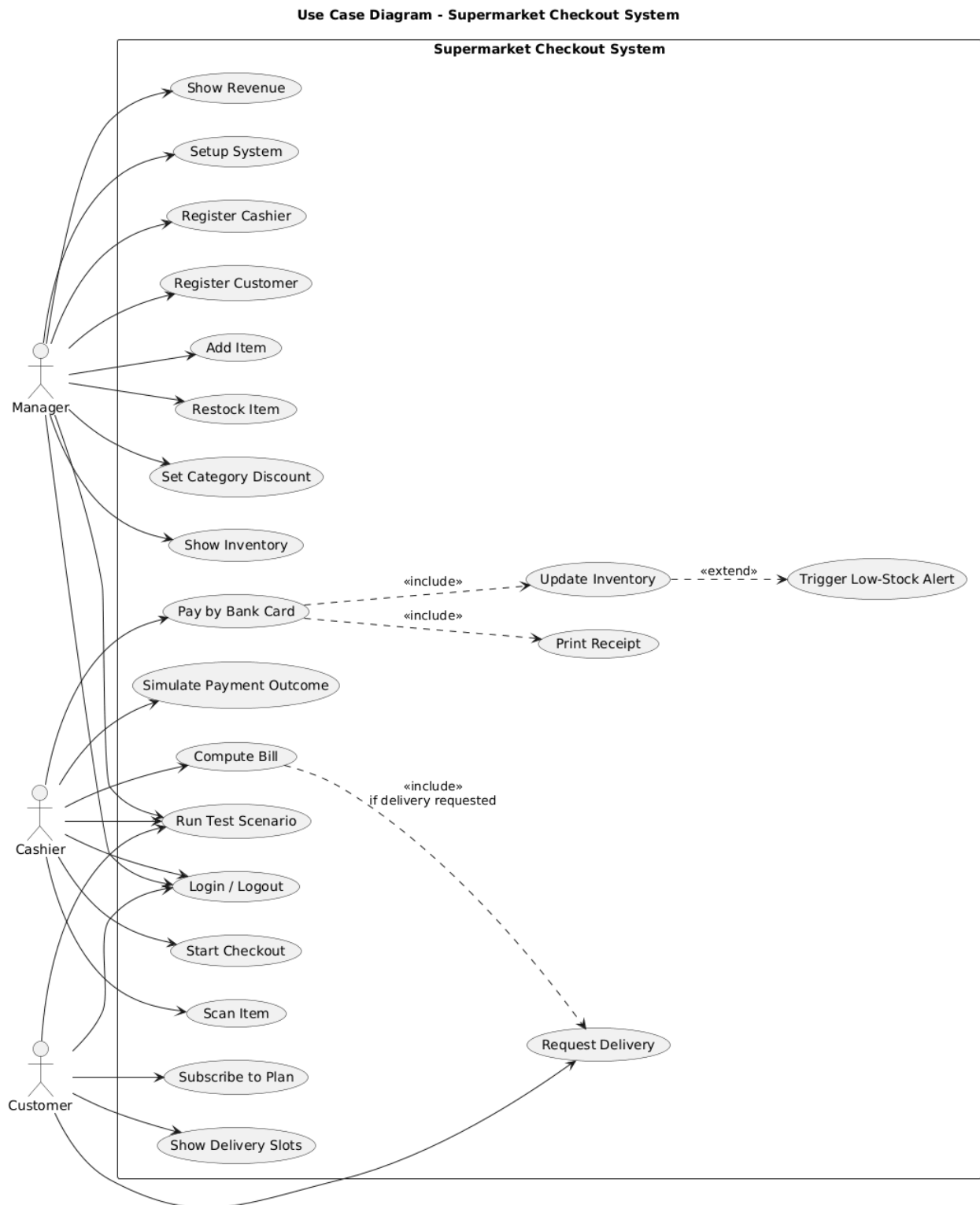


Figure 4: Use case diagram of the supermarket checkout system.

Three actors are identified: the Manager, the Cashier, and the Customer. The Manager handles all administrative operations: system setup, registering cashiers and customers, adding and restocking items, applying category discounts, displaying inventory, and checking revenue. All of these commands are protected by role verification inside **AuthManager**. The Customer can log in, subscribe to a discount plan, and register a delivery request. The Cashier drives the checkout process: opening a session, scanning items, computing

the bill, and launching payment. The diagram also captures key relationships: inventory update and receipt printing are included within the payment use case because they occur only on success; low-stock alert is an extension of inventory update, triggered conditionally; and delivery fee is included in bill computation only when a delivery was previously requested.

Sequence Diagrams

5.3.1 Checkout and Payment

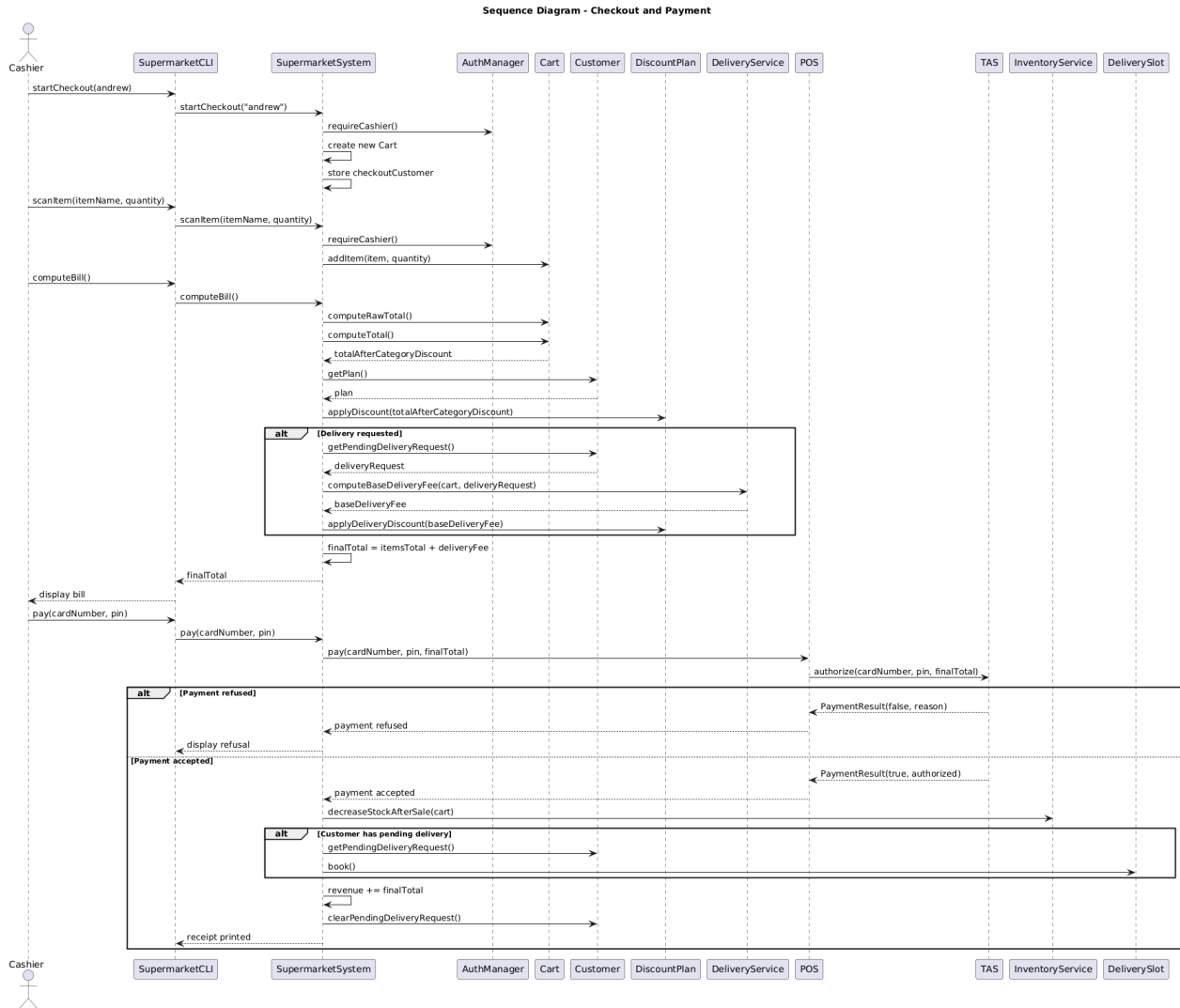


Figure 5: Sequence diagram for the checkout and payment flow.

The checkout sequence begins when the cashier calls `startCheckout`, which creates a new `Cart` and associates it with the selected customer. Item scanning adds `CartLine` entries to the cart. When `computeBill` is called, the system applies category discounts, then retrieves the customer's `DiscountPlan` and calls `applyDiscount`. If a delivery was requested, `DeliveryService` computes the base fee and the plan applies `applyDeliveryDiscount`. Payment is delegated to the POS, which contacts the `TransactionAuthorizationSystem` to validate the PIN and check the account balance. On success, `SupermarketSystem` calls `decreaseStockAfterSale`, adds the amount to revenue, clears the pending delivery request, and prints the receipt. On failure, the cart remains active and inventory is untouched.

5.3.2 Inventory Update and Low-Stock Alert

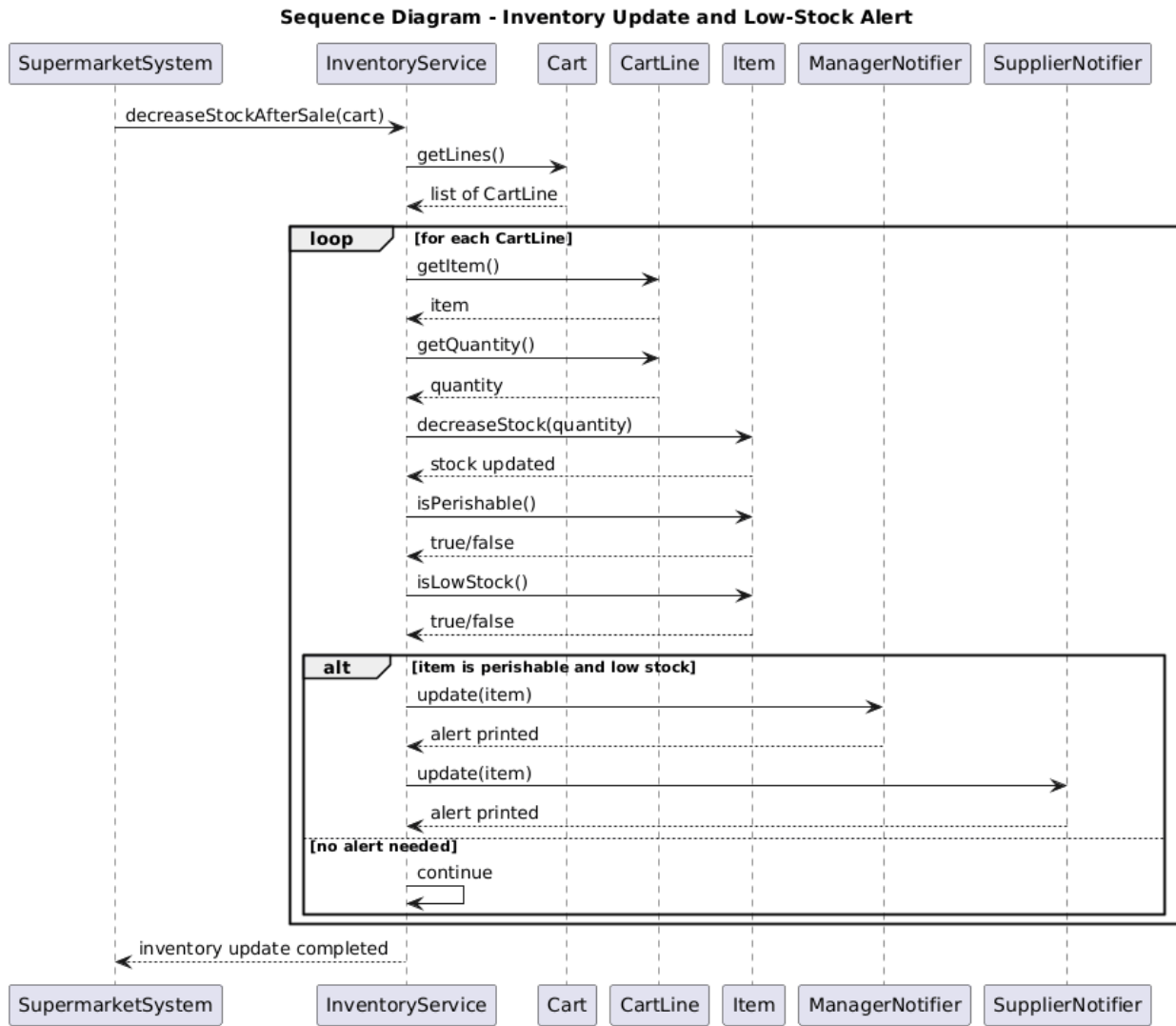


Figure 6: Sequence diagram for inventory update and low-stock alerts.

After a successful payment, `SupermarketSystem` calls `decreaseStockAfterSale` on `InventoryService`, passing the current cart. The service iterates over each `CartLine`, decrements the item stock by the purchased quantity, and then evaluates the two alert conditions: the item must be perishable (meat or dairy) and its updated stock must be at or below its configured threshold. When both conditions are met, `notifyObservers(item)` is called, which dispatches `update(item)` to each registered observer. `ManagerNotifier` and `SupplierNotifier` each handle the notification independently, without any knowledge of the other.

Configuration & Testing

Configuration File (`my_supermarket.ini`)

The configuration file `my_supermarket.ini` is automatically loaded at system boot. In `Main.java`, the system explicitly checks `if (new File("my_supermarket.ini").exists())` and, if found, passes the file to `runTest` before the interactive loop begins. This means the system arrives at its first interactive prompt already fully configured, without requiring any manual setup commands.

The file populates the three mandatory item categories — `meat`, `dairy`, and `fruit-and-vegetables` — with a default catalogue of items at configured prices, weights, and initial stock levels. It registers a default cashier and a default customer, sets up the `prime` and `platinum` discount plans, and pre-registers test bank cards. The resulting state is equivalent to what the interactive `setup` command produces, ensuring that every test scenario starts from a clean, reproducible baseline.

Test Scenarios

6.2.1 `testScenario1.txt`

This scenario covers the end-to-end checkout flow and validates the most critical business rules of the system. It is executed by running the following command from the project root:

```
runTest testScenario1.txt
```

Happy path. The scenario logs in as a customer, subscribes to the prime plan, then logs in as the cashier and opens a checkout session. It scans several items, including `kafta` (a perishable meat product), reducing its stock from 5 to 4 to 0 across successive purchases. `computeBill` is called and the output is verified to reflect the stacked category and prime plan discounts, as well as a dynamic delivery fee for the selected `slot`. Payment succeeds via a registered test card. The system decrements the inventory: once `kafta` reaches 0, the `ManagerNotifier` and `SupplierNotifier` fire their low-stock alerts.

Unhappy path. A second checkout session calls `simulatePayment INSUFFICIENT_FUNDS` before `pay`. The payment is refused by the `TransactionAuthorizationSystem`, and the scenario verifies that `decreaseStockAfterSale` is never called: inventory remains unchanged and no receipt is printed.

JUnit Test Suite

The JUnit test suite is located in the `testmodel` package. Each domain class has a dedicated test class covering all significant methods (getters and setters are excluded per the project rubric). The suite is run through the standard Maven test target or directly from any Java IDE.

Key test coverage highlights are presented in the following table:

Test Class	Method Under Test	Scenario
<code>DeliveryServiceTest</code>	<code>deliveryAbove50KgIsRefused()</code>	Weight above 50 kg throws <code>IllegalStateException</code>
<code>DeliveryServiceTest</code>	<code>deliveryUnder10Kg...</code>	Correct base fee computed for standard delivery
<code>SuperMarketSystemTest</code>	<code>cashierCannotAddItem()</code>	Role check throws <code>IllegalStateException</code>
<code>SuperMarketSystemTest</code>	<code>loginWithWrongPasswordFails()</code>	Invalid login throws <code>IllegalArgumentException</code>
<code>SuperMarketSystemTest</code>	<code>primePlanIsAppliedInSystem...</code>	Prime 20% discount applied when total $\geq 50\text{€}$
<code>SuperMarketSystemTest</code>	<code>deliveryFeeIsReduced...</code>	Prime 50% delivery discount applied successfully
<code>SupermarketCLITest</code>	<code>cliHandlesUnknownCommand...</code>	CLI rejects bad commands without crashing
<code>DeliveryRequestTest</code>	<code>deliveryRequestStoresAddress...</code>	<code>DeliveryRequest</code> correctly stores address, distance, and slot

Workload Split

Class / Task	Design	Code	UML	JUnit
SupermarketSystem	Nohra	Nohra	Nohra	Nohra
SupermarketCLI	Dakdouki	Dakdouki	Dakdouki	Dakdouki
AuthManager	Dakdouki	Dakdouki	Dakdouki	Dakdouki
Cart / CartLine	Nohra	Nohra	Nohra	Nohra
Item / Category	Nohra	Nohra	Nohra	Nohra
DiscountPlan Strategy	Nohra	Nohra	Nohra	Nohra
NormalPlan / PrimePlan / PlatinumPlan	Nohra	Nohra	Nohra	Nohra
POS / TAS	Dakdouki	Dakdouki	Dakdouki	Dakdouki
PaymentResult / PaymentOutcome	Dakdouki	Dakdouki	Dakdouki	Dakdouki
DeliveryService / DeliverySlot	Dakdouki	Dakdouki	Dakdouki	Dakdouki
DeliveryRequest	Dakdouki	Dakdouki	Dakdouki	Dakdouki
InventoryService / StockObserver	Nohra	Nohra	Nohra	Nohra
ManagerNotifier / SupplierNotifier	Nohra	Nohra	Nohra	Nohra
my_supermarket.ini / testScenario1.txt	Dakdouki	Dakdouki	—	—
testScenario2.txt	Nohra	Nohra	—	—
Use Case Diagram	Dakdouki	—	Dakdouki	—
Sequence Diagrams	Nohra	—	Nohra	—
Class Diagram	Nohra	—	Nohra	—
Report	Nohra / Dakdouki	—	Nohra / Dakdouki	—
README / Execution Instructions	Dakdouki	Dakdouki	—	—

Conclusion

This project delivered a fully functional Java supermarket checkout system satisfying all core requirements (R1–R4) and all advanced extensions (R5–R10). The system correctly handles multi-category checkout, stacked discount plans, weight- and distance-based delivery fees, dynamic time-slot logistics, real-time inventory management, and Observer-based low-stock alerts, all accessible through a clean command-line interface.

The most important lesson from this project is that applying strict object-oriented design principles early in the development cycle significantly reduces the cost of implementing complex features later. When the Strategy pattern was in place from the start, satisfying R5b and R6b required no changes to existing classes. When the Observer pattern was established for inventory, adding a second notification target (**SupplierNotifier**) took only minutes. The decision to extract **AuthManager** as a dedicated class made role-based access control straightforward to implement and easy to verify through JUnit without any ceremony.

The main limitations of the current implementation are its in-memory storage model and the absence of real geolocation for delivery distance. In a production version, these would be the two areas requiring the most significant investment: a JDBC-backed persistence layer to replace the **HashMap** stores, and a Maps API integration to replace the hardcoded `distanceKm = 10.0` value in `requestDelivery()`. Concurrency support for multi-cashier deployments would be the third natural extension.

Overall, the architecture is extensible, testable, and well-aligned with the requirements of the project rubric. The combination of a disciplined package structure, two applied GoF patterns, comprehensive JUnit coverage, and a fully automated test scenario provides a solid foundation for any future evolution of the system.

Appendix A: Complete Class Diagram

